{EPITECH}

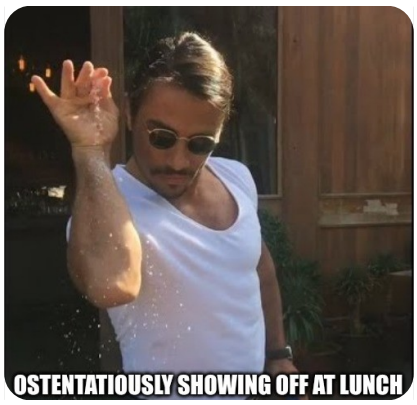
KUBEQUEST_

< SAILING THROUGH THE CLOUDS />

KUBEQUEST

Welcome to an exhilarating journey into the world of cloud computing and containerization!
You are part of a dynamic team at a forward-thinking company, poised to explore the robust capabilities of Kubernetes.

Your mission is to revolutionize the company's containerization infrastructure, showcasing your skills in software engineering and system management.



Objectives

Your primary task is to design and deploy a fully-equipped Kubernetes cluster, not only to host and present application, but also with management tools and administration components.

Below are the goals you need to achieve, with some *tool suggestions* as starting points :

- ✓ an internal **load balancer** to expose apps and services of your infra (*nginx-ingress*) ;
- ✓ a user-friendly **dashboard** to access and manage the containers (*kubernetes-dashboard*) ;
- ✓ a **monitoring stack** to collect and visualize system and apps metrics (*kube-prometheus*) ;
- ✓ a **GitOps** repository with all these reusable components manifests (*kustomize*) ;
- ✓ a **logging stack** to collect and visualize some logs (*loki*).

After that, convert and deploy an application which was actually deployed using *docker-compose*:

- ✓ create an **helm chart** for this application, and use official one for database (*helm*) ;
- ✓ create a **GitOps** repository to deploy this application (*kustomize*) ;
- ✓ **automate** deployment and verify application status.

You can find the provided application on Gandalf in the module.

During this experimentation, implement all the ”best practices” in a Kubernetes environment :

- ✓ define resources **limits and requests** on CPU and ram consumption ;
- ✓ use **secrets** to host sensible data ;
- ✓ **label** all your resources, following kubernetes recommandations ;
- ✓ add **redundancy** on your application, with multiple replicas and affinity rules, at least for ”application” containers ;
- ✓ add **persistent storage** for your database, and implement usual backup system using Kubernetes.

You'll also have to add security and control :

- ✓ add a **validating webhook** that can validate and control request (opa) ;
- ✓ add **authentification** to your Kubernetes API and your different tools (dex oauth-proxy).

Infrastructure

Each team is provided with dedicated compute resources hosted on **Amazon Web Services (AWS)**, consisting of **4 Linux virtual machines** :

- ✓ `kube-1` — Kubernetes node (control plane + worker) ;
- ✓ `kube-2` — Kubernetes node (worker) ;
- ✓ `ingress` — Ingress controller, exposes services externally ;
- ✓ `monitoring` — Monitoring stack, metrics, logs and dashboards.

The two Kubernetes nodes form your cluster: `kube-1` acts as the control plane and first worker, while `kube-2` provides additional compute capacity. The dedicated `ingress` node handles all inbound traffic routing via an ingress controller, and the `monitoring` node hosts your observability stack (Prometheus, Grafana, Loki).

You can provision and configure your machines using Terraform with the AWS provider:
https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/instance

Please note that all virtual machines are automatically shut down every evening to conserve resources and minimize costs. However, you can restart them at any time if needed.

Delivery

Push your configurations into a repo.

Documentation is mandatory, especially the containers setup steps and commands.

Defense

Concerning the final defense of your project:

1. before presenting, start a fresh new Kubernetes cluster in the cloud, with many nodes ;
2. while presenting, deploy the services into the orchestrator using nothing more than `kubectl apply -f ... -f ... -f ...`, `kustomize...`, `helm ...` ;
3. in order to demonstrate some features, such as auto-scaling, execute some live scripts or commands you wrote beforehand in order to send lots of requests to your applications ;
4. demonstrate a full deployment process and a broken deployment with automatic rollback.

> Feel free to enrich the applications code with some memory leaks, loops consuming CPU, or anything that could lead to errors and container failure.

Bonuses

You may also want to add some extras, such as:

- ✓ an encryption (with *Let's Encrypt* and *cert-manager*) ;
- ✓ *argoCD* to manage your components and GitOps repositories ;
- ✓ setup multi-tenant and authentification to *prometheus* and *loki*, via *kube-rbac-proxy* ;
- ✓ evaluate Helm artefacts security ;
- ✓ define the orchestrator access permissions ;
- ✓ reinforce the network access ;
- ✓ create some lightweight Docker images ;
- ✓ guarantee zero-downtime deployment (successful HTTP request during deployment) ;
- ✓ pull Docker images from a private registry ;
- ✓ ...

v 1.0.2

{EPITECH}